

Sprint5 Study Guid

Focus areas:

OOAD: Object-Oriented Analysis and Design

UML: Unified Modeling Language

SOLID: Principles for maintainable object-oriented design

TDD: Test-Driven Development

JUnit 5: Java unit-testing framework

Mockito: Java mocking framework for testing

Java/Python: Comparing concepts, syntax, and typing between Java and Python

GenAI Review: Critically evaluating AI-generated code and suggestions

1. High-Value Concepts to Know

Concept	What to remember
Encapsulation	Hide mutable state; expose controlled operations that preserve business rules and object invariants.
OOAD	Treat the first object model as a hypothesis. Class boundaries and responsibilities can change as the team learns.
SRP - Single Responsibility Principle. The S in SOLID	A class should have one focused responsibility / one main reason to change. One class, one job.
OCP - Open/Closed Principle. The O in SOLID	Prefer adding new implementations over modifying stable existing code. Open to ADD, Closed to CHANGE.
LSP - Liskov Substitution Principle. The L in SOLID	A subtype must honor the behavioral expectations of its base type. A subclass should be usable anywhere its parent class is expected without breaking expected behavior.
ISP - Interface Segregation Principle. The I in SOLID	Clients/classes should not depend on methods or capabilities they do not need. Keep interfaces small and focused.
DIP - Dependency Inversion Principle. The D in SOLID	High-level code should depend on abstractions, with concrete implementations supplied from outside. Depend on interfaces/abstractions, not concrete classes.
TDD	Red → Green → Refactor. Make the smallest change that passes the current test, then let the next test drive more behavior.
JUnit 5	Use specialized features such as <code>assertThrows</code> , parameterized tests, nested tests, and display names when they improve clarity and coverage.
Mockito	Use mocks to isolate meaningful collaborators; do not mock simply because a class exists.

2. UML Quick Reference

Aggregation — `o--`: A weaker 'has-a' relationship. The part can exist independently of the whole.

Composition — `*--`: Strong ownership. The part is conceptually tied to the lifecycle/meaning of the whole.

Dependency — `..>`: Temporary use, such as receiving an object as a method parameter without retaining it.

Inheritance / Generalization: An 'is-a' relationship between a subtype and a base class.

Sequence alt fragment: Shows mutually exclusive paths, such as accepted vs. rejected order processing.

3. Question-by-Question Review

Question 1

A class exposes a critical property as a public field, allowing callers to modify it directly. You make the field private and provide methods for accessing and modifying the field. What OO principle have you primarily applied?

Study point — Encapsulation: Keep state private and control changes through methods that enforce invariants. Here, callers can read quantity but cannot bypass the rule that quantity must never become negative.

Exam cue: *private state + guarded update → encapsulation*

Question 2

When transitioning from Python to Java. What are some of the similarities, and what are some of the differences?

Study point — Python → Java transition: Core programming ideas such as types, collections, exceptions, and control flow transfer well. Java adds compile-time type checking and more explicit syntax, so thinking first in terms of the Python equivalent can help, while remembering that Java's compiler catches many type mismatches earlier.

Question 3

Two people are modeling the behavior of a Risk Limit where one uses a class and the other uses an attribute. Is there always one objectively correct design? What do you do when disagreements arise?

Note: write down both views and the reasoning, since that disagreement is useful input for later review and refinement

Study point — OOAD is iterative: Class-versus-attribute choices can be legitimately ambiguous. Capture the alternatives and reasoning, then refine the model as requirements, tests, and reviews provide more evidence.

Exam cue: *design disagreement can be useful evidence*

Question 4

Most responsibilities—validation, calculation, execution, and reporting—are placed in one `OrderManager` class, while other classes mainly hold data. What design problem should this raise?

Example of A "God class" design smell, where responsibilities that belong on other objects have been dumped onto one manager class — a problem SOLID (particularly SRP) will formally address later

Study point — God class / SRP smell: A central manager that owns nearly every verb is a warning sign. Responsibilities should usually live with the objects that have the data and rules needed to perform them.

Exam cue: *one manager owns all verbs → God class smell*

Question 5

What are the different types of UML relationship pairings?

Portfolio o-- Instrument → Aggregation: Instruments can exist independently of the Portfolio.

Portfolio *-- Holding → Composition: A Holding is dependent on its Portfolio.

OrderValidator ..> RiskLimit → Dependency: OrderValidator temporarily uses RiskLimit, such as through a method parameter.

Study point — UML relationships: Aggregation represents a weaker 'has-a' relationship; composition represents owned lifecycle; dependency represents temporary use. Inheritance/generalization is for 'is-a' relationships, and interface implementation uses realization notation rather than a dependency.

Question 6

How should a UML sequence diagram show that an order is executed if valid, but rejected with a reason if invalid?

Wrap the diverging behaviour in a single alt fragment with two branches, with one branch showing execution/acceptance and the other showing rejection with a reason

Study point — Sequence diagram alt fragment: Use an alt combined fragment for mutually exclusive paths such as valid vs. rejected. Each branch can show its own messages and outcome.

Exam cue: *either/or workflow* → *alt*

Question 7

Which practices demonstrate effective peer review of UML designs? (Select all that apply.)

If you disagree with a comment on your own design, replying with your reasoning and checking whether it addresses the reviewer's concern, rather than dismissing it outright; Giving an explicit approve-or-change-request decision, with one sentence explaining the reasoning; Leaving at least two specific, constructive, written comments rather than a vague "looks fine"

Study point — Constructive peer review: Good review is specific, written, reasoned, and open to discussion. It should identify actionable concerns and end with a clear approve/change-request decision.

Question 8

FrozenHoldingBad extends Holding but overrides `adjust()` to always throw an exception. Which statements about this design are correct?

Use SOLID to help determine the answer. This is a Liskov Substitution Principle violation, because FrozenHoldingBad cannot be substituted everywhere a Holding is expected without changing correct behaviour; A better fix is for FrozenHolding to hold a Holding as a field (composition) rather than extend it, and simply not expose an `adjust()` method at all

Study point — Liskov Substitution Principle: A subtype should be usable wherever its base type is expected without surprising breakage. If a frozen holding cannot support `adjust()`, composition can model that constraint more honestly than inheritance.

Exam cue: *subtype breaks base expectations* → *LSP*

Question 9

A new DerivativeInstrument must be added without modifying Instrument, Feeable, Order, or existing instrument classes. Why is this good design? (Select all that apply.)

Again use the SOLID principles. Extending Instrument for the new type keeps the polymorphic contract intact, so existing code that works with Instrument or Feeable references continues to work unchanged; This is an

application of the Open/Closed Principle — the system should be open for extension (new instrument types) but closed for modification (existing, already-tested classes stay untouched)

Study point — Open/Closed Principle: Add new behavior by extension rather than changing stable, already-tested code. A new instrument can extend the abstraction and preserve existing polymorphic contracts. Remember the Open/Closed Principle (OCP) — open for extension, closed for modification.

Question 10

A report builder handles fee calculations, report formatting, and file output. You split these responsibilities into separate classes. Which SOLID principle does this demonstrate?

Think of the Single Responsibility Principle

Study point — Single Responsibility Principle: Separate fee aggregation from report formatting so each class has one reason to change.

Exam cue: *split unrelated jobs → SRP*

Question 11

A colleague's OrderExecutor class currently does this: "new ConsoleReportWriter()" is called directly inside execute(), so every order writes straight to the console with no way to redirect or test that output. You suggest changing the constructor to accept a ReportWriter interface parameter instead, and store that. Which of the following are true about this change? (Select all that apply.)

Study point — Dependency Inversion + testability: Inject a ReportWriter abstraction instead of constructing ConsoleReportWriter inside OrderExecutor. This can also be done using an interface where mock implementations can use that interface. The Dependency Inversion Principle is also in play where the OrderExecutor not depends on an injected ReportWriter. This makes output replaceable in tests and removes the high-level class's direct dependency on a concrete writer.

Exam cue: *inject interface → DIP + easier testing*

Question 12

Your team has a fat CsvReportable/ConsoleReportable-style interface design question: a new CsvSettlementReport class only ever needs to produce CSV output, never console output. A colleague suggests implementing both CsvReportable and ConsoleReportable "just in case it's needed later." Which statements correctly explain why this is worth pushing back on in review? (Select all that apply.)

Study point — Interface Segregation Principle: Do not make a class depend on capabilities it does not need. Adding console behavior 'just in case' creates unnecessary coupling and can violate an explicit class-contract test. Don't force your class to depend on methods it doesn't need or use. Remember also, you can use reflections to confirm what methods are implemented by an interface and also see what is public or private.

Question 13

A colleague pastes a messy `OrderCategoriser` class into GitHub Copilot Chat and gets back a polished-looking refactor that renames a few variables, extracts two helper methods, and changes the structure enough to look "cleaner" at a glance. The code compiles, but no one has yet compared the behaviour against the original version. Following this sprint's guidance on critically assessing GenAI suggestions, what should the colleague do before accepting it?

Study point — Critically assess GenAI refactors: Compilation is not enough. Preserve the existing tests, run them unchanged, and inspect each suggested change for readability and behavior preservation before accepting it. Compare the results of the modified class to the original and ensure the results are the same. Ensure behavior is preserved rather than just looking to see if the class looks refactored.

Question 14

You're building a `FeeBandClassifier` under strict TDD for a pricing service that will later be extended with additional bands. For the very first red-green cycle, the only test says `classify(1000)` should return "STANDARD". A colleague reviewing your commit is surprised that your green implementation just returns "STANDARD" unconditionally, with no conditional logic at all. Is this a shortcut you're getting away with?

Study point — TDD: smallest green implementation: In red-green-refactor, the green step is the smallest implementation that satisfies the current failing test. A later test should force additional logic when it is actually required. The implementation can be as simple as returning a desired value.

Exam cue: first green can be intentionally simple

Question 15

You're writing a JUnit 5 test suite for `RiskLimitChecker.canAcceptOrder(currentPortfolioValue, orderValue, riskLimit)`, which returns true/false based on whether the order fits within the risk limit, and throws `IllegalArgumentException` for `orderValue <= 0` or `riskLimit < 0`. Which of the following are genuinely good uses of JUnit 5 features for this class, as opposed to using `@Test` and `assertEquals` for everything?

Study point — JUnit 5 features: Use `assertThrows` for exception behavior, `@Nested/@DisplayName` to document behavioral groups, and `@ParameterizedTest/@CsvSource` for multiple input combinations and boundary cases.

Question 16

A colleague's class, `OrderExecutor`, depends on both an `Instrument` and a `ReportWriter`. In a JUnit test, they use real `BondInstrument` and `InMemoryReportWriter` objects rather than mocks. A teammate suggests rewriting the test with Mockito's `@Mock` and `verify(...)` instead. What are the trade-offs being weighed? \

Study point — Mocks vs. real collaborators: Real collaborators exercise a broader slice of behavior but make failures less isolated. The result is you turn your unit tests into more of an integration test scenario. This means a red result could surface as a result of code in another layer. Mocks can isolate `OrderExecutor`. Mocking is a tool, not a requirement—simple collaborator-free classes often need no mocks. Also prepping a proper test bed can help with mock implementations.

Question 17

While wiring together already-tested classes in `OrderProcessingEngine`, you notice a tempting shortcut: instead of calling `Instrument.calculateFee()`, you could copy the fee calculation into the orchestrator and tweak it for one edge case that the finance team keeps asking about. The team argues that it would be "quicker" and would avoid one method call. What should this urge signal to you?

Study point — Keep orchestration thin: An orchestrator should coordinate existing domain behavior rather than duplicate it. Copying fee logic creates multiple sources of truth and weakens the value of earlier tests. If you find yourself copying code, it usually means there's an opportunity for inheritance or usage of polymorphism. The orchestrators job is to cording and not to handle business functionality.

Exam cue: *orchestrator coordinates; domain objects own rules*

Question 18

In the order-processing flow, `OrderProcessingEngine.process()` looks up a `Client`, validates a proposed order via `OrderValidator`, and - only if valid - calculates a fee, updates the `Holding`, and adjusts the `Portfolio`'s total value. In a design review, someone asks what should happen when the order is rejected after validation fails. What should the correct behaviour be, per the mission's requirements?

Study point — Rejected-order invariant: If validation fails, record the rejection and reason, but do not mutate holdings or portfolio value. There should be no changes to the client's portfolio landholdings should be left unchanged.

Exam cue: *rejection must cause no domain-state mutation*

Question 19

Comparing a first-draft object model against the class actually built, a colleague finds that "`RiskLimit`" ended up as a plain attribute on `Client` rather than its own class, contradicting their original worksheet entry. What are some reasonable, legitimate causes for a design decision changing between an early OOAD draft and the final build?

Study point — Design evolves with evidence: Early OOAD is a draft. Peer review and TDD can reveal that a proposed class boundary does not fit the real behavior, so the final model may legitimately differ. In some cases TDD will reveal changes need to be made to class boundaries or hierarchies.

Question 20

You are building a Java application that needs to Store trade events in order and allow duplicates, Store processed instrument IDs without duplicates, and Look up a holding quantity by instrument symbol.

Study point — Know which Java collections are best suited for these requirements? Java framing for Python developers: The concepts are largely transferable in terms of collections, but Java is more explicit and statically typed. Use Python analogies to help understand intent, then account for Java's compile-time rules. Good Tips to remember:

List → maintains order and allows duplicates.

Set → ensures values are unique.

Map → stores key-value pairs and provides efficient key-based lookup.

4. Final Review Checklist

- ☐ Can I distinguish encapsulation from the SOLID principles?
- ☐ Can I identify a God class and explain why SRP is relevant?
- ☐ Can I explain aggregation, composition, dependency, inheritance, and an alt sequence fragment?
- ☐ Can I explain OCP, LSP, ISP, and DIP using the trading-system examples?
- ☐ Can I describe red-green-refactor and why the first green implementation may be deliberately minimal?
- ☐ Can I choose `assertThrows`, `@Nested/@DisplayName`, and `@ParameterizedTest` appropriately?
- ☐ Can I explain when mocks improve isolation and when real collaborators are reasonable?
- ☐ Can I explain why orchestration code should not duplicate domain rules?
- ☐ Can I state the rejected-order invariant: record rejection, change no holdings/portfolio state?
- ☐ Can I explain why Java concepts transfer from Python while static typing changes when errors are caught?

5. One-Minute Memory Map

E-SOLID: Encapsulation protects state. SRP separates jobs. OCP extends without modifying. LSP preserves substitutability. ISP avoids unused interface obligations. DIP injects abstractions.

Testing: TDD grows behavior one failing test at a time; JUnit features improve expression and coverage; Mockito is for isolation, not automatic use.

Design: OOAD evolves. UML communicates structure and flow. Orchestrators coordinate; domain objects own business rules.